Design layer

the whole layer, as one document

Contents

1 Design Layer 4

1.1 Foundation 4
1.2 System at a glance 5
1.3 Progressive breakdown 5
1.4 End-to-end walkthrough 6

Glossary 6

Design Layer

1 Design Layer

Design Layer helps a project explain its architecture to people and code agents from one authored source. It combines documentation guidance with a checked Typst library, a PDF renderer, a Markdown projector, and a document gate. The project being described remains responsible for its architecture.

1.1 Foundation

Shared understanding with explicit limits.

GOAL

Orient a newcomer

Explain purpose, boundaries, principal abstractions, and their cooperation in a form that people can read as PDF and agents can read as Markdown.

NO-GOAL

Choose the host architecture

This repository guides documentation. It does not choose the host's architecture, prescribe its implementation, or prove that its code satisfies the documented intent.

INVARIANT

Reject undefined meaning

An illegal declared value or unresolved semantic reference fails validation. The Markdown projector rejects unsupported content instead of silently dropping it.

mechanism

PRINCIPLE

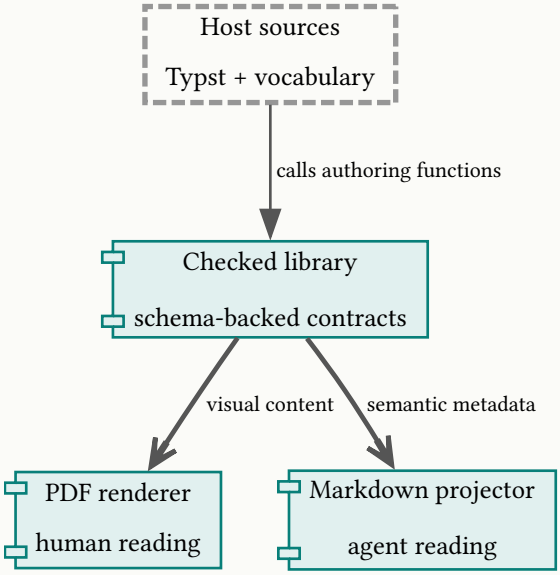
Keep implementation authority in code

The layer explains the system at architectural depth. Code remains authoritative for implemented detail; established owner intent supplies goals and rationale. A passing document gate establishes well-formedness.

1.2 System at a glance

One source serves two reading modes.

ALTITUDE L3 · COMPONENTS The document pipeline



LEGEND



The host supplies authored content. The library checks it and exposes visual output or semantic metadata. The PDF renderer and Markdown projector consume those respective views. The separate gate also checks references across the host's artifacts.

Host boundary. A host owns its `design.typ`, glossary `CONTEXT.typ`, and supporting Markdown records under `docs/`. Its code is not an input to the renderer. A human or agent uses the documentation skills to select supported architectural claims before authoring them.

Tool boundary. Nix packages the scripts, schema, library, Typst compiler, fonts policy, and vendored drawing dependencies. A host pins that bundle. The public apps prepare a disposable `.render` library beside the layer; source files call its functions directly. There is no Markdown authoring conversion between those calls and their contracts.

State boundary. Authored sources and the host's committed PDF persist. Library copies, compile workspaces, and agent Markdown are derived outputs. The system owns no application database or long-running service.

1.3 Progressive breakdown

Responsibilities follow the evidence each part can see.

Documentation guidance selects what a newcomer needs and where explanation stops. `skills/document-design/` owns that judgment; the syntax skill owns exact notation. These skills accompany the same bundle as the tools.

The checked library reads the schema at compile time. A block contract gives a recurring concept one structured authoring call and one visual representation. Schema vocabularies and semantic tokens keep those calls consistent. Library code is handwritten; assembling a library copies it with its schema rather than generating a second implementation.

Validation has three scopes. Individual calls check their own values. Compile-local trails collect facts across calls, then document assertions check the completed scope; this can detect an omitted statement as well as a malformed one. Artifact integrity resolves links, glossary ownership, and declared file homes across the repository.

An Invariant rejects invalid meaning. A Guideline reports a structural or style expectation without blocking an ordinary render. Strict mode promotes guidelines to errors. Reports stay outside the PDF so readers see the authored design rather than authoring diagnostics.

The PDF renderer discovers the layer, combines its chapters and vocabulary, and supplies the shared document shell. Rendering and validation use the same Typst calls. The aggregate reads guideline metadata and reports it to the author.

The Markdown projector queries Semantic metadata emitted by those calls. It preserves decision-bearing fields and relationships while omitting presentation. Complete output and section output use the same projection. A manifest gives section addresses and a digest; a caller can require that digest when loading sections from one snapshot.

The document gate regenerates and compares the committed PDF, checks semantic token coverage, and checks cross-artifact integrity. The gallery and self-tests separately exercise the tools, including rejected inputs. This layer exercises integration; the self-tests prove individual contracts.

1.4 End-to-end walkthrough

An architecture change becomes a shared reading surface.

Author. An author inspects the host's code and established intent, then updates only the architectural claims affected by the change. Vocabulary has one owning glossary. Detailed implementation remains in the host's code.

Render. The render app prepares the selected library and compiles the layer. Invalid values or references stop validation. Guidelines are reported; they block only when strict mode is requested. A successful render writes the aggregate PDF.

Project. The context app queries the same authored layer and emits Markdown. For a large layer, an agent first reads the manifest and then requests the relevant section subtree with its expected digest.

Check. The author inspects both reading forms and runs the gate. A stale PDF fails freshness; a broken cross-artifact link fails integrity. Passing these checks does not establish that the host implements its claims.

Representation limits. Semantic diagrams preserve nodes and relationships in Markdown. A custom figure with a raw `typst` body becomes a drawing in PDF and a code fence in Markdown. Its caption or adjacent prose must state its meaning. Evaluated custom figure bodies and embedded SVGs cannot currently be projected to Markdown and fail that projection.

Reproducibility boundary. The bundle fixes renderer dependencies and excludes system fonts. Compilation uses vendored packages without fetching them at render time. Obtaining the Nix inputs and building the bundle is a separate provisioning step. A bundle update can change output, so hosts must re-render before freshness can pass.

Where to investigate. `lib/designlib.typ` maps the library responsibilities; `schema/design-schema.json` declares the vocabulary. `scripts/design-aggregate` and `scripts/design-context` own the two reading outputs. `scripts/layer-integrity` owns cross-artifact checks. `flake.nix` assembles the bundle and public apps. The gallery and self-tests provide executable examples and failure cases.

Glossary

Every term the layer declares, with the context that owns it.

Guideline

owned by root

A structural or style expectation that is reported during ordinary rendering. Strict mode makes a guideline violation an error.

Invariant

owned by root

A rule whose violation makes the document model invalid. Its validation failure prevents a successful check.

Semantic metadata

owned by root

Structured content emitted by the authoring functions in projection mode. It carries document meaning independently of visual layout.